

Instilling Physical Dynamics in a model: Predicting coefficient of restitution via Deep Learning

Yash Sharma

Abstract

I try to present a physics-informed deep learning framework that learns to predict the coefficient of restitution (CoR) of collisions directly from raw pixels, without access to any explicit physics equations during inference. Given a video of two objects colliding, the model predicts a scalar CoR value in $[0, 1]$ representing the ratio of relative velocities before and after collision. I train four distinct temporal architectures on a self-generated dataset of 3,300 collision videos and evaluate each on in-distribution performance, out-of-distribution generalization, and latent space interpretability. Our best model, a 3D Convolutional Neural Network, achieves a test R^2 of **[0.9630]** and mean absolute error of **[0.0483]**. We further demonstrate that the learned latent space implicitly encodes Newton's law of momentum conservation with **[6.87]%** relative error despite never being trained on it. Different independent analyses consistently confirm that the encoder organizes its internal representation around physical quantities rather than visual appearance.

Introduction

While contemporary computer vision models demonstrate remarkable proficiency in object recognition, edge detection, and scene classification, they remain fundamentally deficient in reasoning about the underlying physical forces and material properties that govern observed interactions.

I address this gap by framing physics as a regressing problem over a property of collision which is Coefficient of Restitution (CoR) which is defined as -

$$CoR = |v_{b_after} - v_{a_after}| / |v_{a_before} - v_{b_before}|$$

where v is the velocity of the objects. A CoR of 0.0 means a perfectly inelastic collision (maximum energy loss) and a CoR of 1.0 means a perfectly elastic collision (full energy conserved). Predicting this quantity requires the model to learn the before and after velocities of the objects through raw pixels. CoR prediction is a quantitative physics problem that requires encoding the underlying mechanical properties of the collision.

- **Section 1 - Dataset**
- **Section 2 - Models and Training Procedure**
- **Section 3 - Experimental Results**
- **Section 4 - Physics understanding Analysis**
- **Section 5 - Conclusion**

1. Dataset

I model 1D elastic and inelastic collisions between two objects.
The chosen physical output is the Coefficient of Restitution (CoR), defined as:

$$e = |v_{b'} - v_{a'}| / |v_a - v_b|$$

where v_a , v_b are pre-collision velocities and $v_{a'}$, $v_{b'}$ are post-collision velocities.

Why CoR:

- It is a single scalar that completely characterises the energy dissipation behaviour of a collision
- It is continuous in $[0, 1]$ making it a regression target
- Ground truth is directly controllable in simulation via pymunk's elasticity parameter
- A model that predicts CoR correctly from pixels must have encoded the velocity change across the collision — genuine physics understanding

The chosen physical interaction would then require videos which would then be processed into individual frames spread uniformly across the video to reduce training time.

The videos would contain 2D objects interacting with each other.

The physical interaction (Collision) is constrained to be only in 1D as a control setup. The purpose of this decision is to find which model is able to learn the physics and also then to find which model can follow that physics in another environment.

1.1 Generation

Collision Synthetic Dataset of 3,300 samples is generated using a python library named “Pymunk”. All simulations are in 2D and run in a $1,000 \times 1,000$ unit canvas with no gravity and no damping, representing frictionless vacuum conditions. Collisions are rendered at 512×512 pixels using pygame and saved as MP4 videos at 60 Hz for 120 frames (2 seconds of real-time physics).

Each sample has 2 square objects colliding head-on along the horizontal axis. The CoR is set as the elasticity parameter in pymunk, which directly controls the ratio of relative velocities before and after collision according to the CoR equation. This gives exact ground-truth CoR labels for every sample.

Both the objects have equal mass (1.0 each) and similar sizes, and are restricted to collide head-on along horizontal axes. Rotation of the objects has been disabled

1.2 Preprocessing

Videos are preprocessed before training: each MP4 is decoded once, 20 frames are sampled at uniform intervals, each frame is resized to 64×64 pixels, and the result is saved as a compressed .npz file (shape: $20 \times 64 \times 64 \times 3$, uint8). Direct training on MP4 was found to slow down DataLoader and make training impractical. Preprocessing helps reduce this time cost to a one-time operation of approximately 5 minutes for all 3,300 videos.

1.3 Out of Distribution (OOD) Test Dataset

Two additional test sets are generated for generalization evaluation, each containing 200 samples. Neither is used for training or hyperparameter selection

- Unequal Mass - The masses of both the objects are not the same. Their masses differ. The CoR formula remains the same but there is asymmetric transfer of momentum between the objects.
- Off Centre Collision - The objects now do not experience head on collision. One of the objects is slightly pushed away from the central axis.

2. Models and Training

2.1 Task Formulation

Given a sequence of frames $F = \{f^1, \dots, f^{20}\}$ where each $f^i \in \mathbb{R}^{3 \times 64 \times 64}$, the model produces three simultaneous predictions:

- **CoR prediction:** $\bar{e} \in [0, 1]$, the coefficient of restitution
- **Final velocity of object A:** $\hat{v}^a \in \mathbb{R}$, the post-collision horizontal velocity
- **Final velocity of object B:** $\hat{v}^b \in \mathbb{R}$, the post-collision horizontal velocity

The three outputs are not independent and are constrained by the CoR equation. This constraint is enforced through physics consistency loss described later in the report.

2.2 Architecture Design

- **Architecture 1 (CNN + LSTM):**

The intuition to select this architecture is sequential processing. CNN processes the space of the simulation, LSTM then sees time sequentially. Limitation: LSTM processes frames left-to-right — it sees the collision and then "forgets" early frames gradually. Long-range dependencies between pre-collision and post-collision frames may be weakly captured.

- **Architecture 2 (CNN + Transformer):**

The intuition to select this architecture is attention over all frames simultaneously. Unlike LSTM which reads sequentially, the Transformer sees all 20 frames at once and learns which frames matter most for CoR prediction. The collision moment frame should receive the highest attention weight. Limitation: Transformers need more data than LSTMs to generalise well — 3300 samples is on the small side for a Transformer.

- **Architecture 3 (3D CNN):**

The intuition here is joint spatiotemporal processing (handling space and time simultaneously). Instead of separating space (CNN) and time (LSTM/Transformer), a 3D CNN learns filters that operate across both spatial dimensions and the time dimension simultaneously. A 3D filter of size (3, 3, 3) detects patterns that span 3 consecutive frames and 3×3 spatial regions — it can directly detect motion.

- **Architecture 4 (Resnet18 + LSTM):**

The intuition here is transfer learning. ResNet18 pretrained on ImageNet already knows how to detect edges, shapes, object boundaries — features that are useful for tracking objects in your collision videos. Instead of learning these from scratch on 3300 samples, we start from a model that learned them from 1.2 million images and fine-tune them. Limitation: ImageNet features are tuned for natural images — white background synthetic physics simulations are visually very different, creating a domain gap.

2.2 Loss Function

$$L_{\text{total}} = L_{\text{CoR}} + \lambda_v(L_{\text{vel_A}} + L_{\text{vel_B}}) + \lambda_p \times L_{\text{consistency}}$$

$$L_{\text{CoR}} = \text{MSE}(\text{CoR}_{\text{pred}}, \text{CoR}_{\text{true}})$$

$$L_{\text{vel_A}} = \text{MSE}(\text{vel_A}_{\text{pred}}, \text{vel_A}_{\text{true}})$$

$$L_{\text{vel_B}} = \text{MSE}(\text{vel_B}_{\text{pred}}, \text{vel_B}_{\text{true}})$$

$$L_{\text{consistency}} = \text{MSE}(|\text{vel_B}_{\text{pred}} - \text{vel_A}_{\text{pred}}| / |\text{vel_A}_{\text{init}} - \text{vel_B}_{\text{init}}|, \text{CoR}_{\text{pred}})$$

$$\lambda_v = 0.5, \lambda_p = 0.3$$

We modify the loss function to not only take into account the loss committed in predicting CoR and final velocities but also to penalize the physics inconsistency. The consistency loss penalises the disagreement between implied and predicted CoR, forcing the encoder to develop a latent representation where all three heads tell the same physical story. This modification in the loss function acts an important way to make the model learn physics.

Hyperparameter	Value
Optimiser	Adam
Initial LR	1e-3
LR Schedule	StepLR, $\times 0.5$ every 10 epochs
Epochs	40
Batch size	16
Train/test split	80/20, seed=42
λ_v	0.5
λ_p	0.3

ResNet architecture is trained differently. For the first 10 epochs, its parameters are frozen and after that they are unfrozen.

3. Experimental Analysis

3.1 Test Scores

Accuracy on IN Distribution Data

Models	MAE	R² Score
CNN + LSTM	0.0455	0.9698
CNN + Transformer	0.0440	0.9694
3D CNN	0.0483	0.9630
ResNet18 + LSTM	0.0434	0.9708

From the in-distribution test, Architecture 4 seems to be the best predictor although accuracy of other architectures is not very different. Performance of all architectures is somewhat similar on the in distribution test.

Accuracy on OUT OF Distribution Data

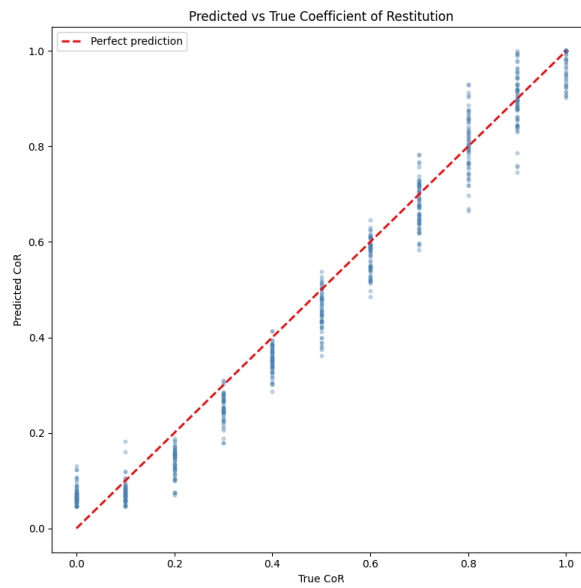
Models	Unequal Mass MAE	Off Centre MAE	Unequal Mass R² Score	Off Centre R² Score
CNN + LSTM	0.1311	0.0602	0.5699	0.9404
CNN +Transformer	0.1207	0.0726	0.7506	0.9195
3D CNN	0.0940	0.0706	0.8378	0.9233
ResNet18 + LSTM	0.0549	0.1749	0.9457	0.3996

From the out-of-distribution data, it is clear that the models struggled to predict the CoR, hence not being able to learn physics properly. The best performing model was Architecture 3 - 3D CNN.

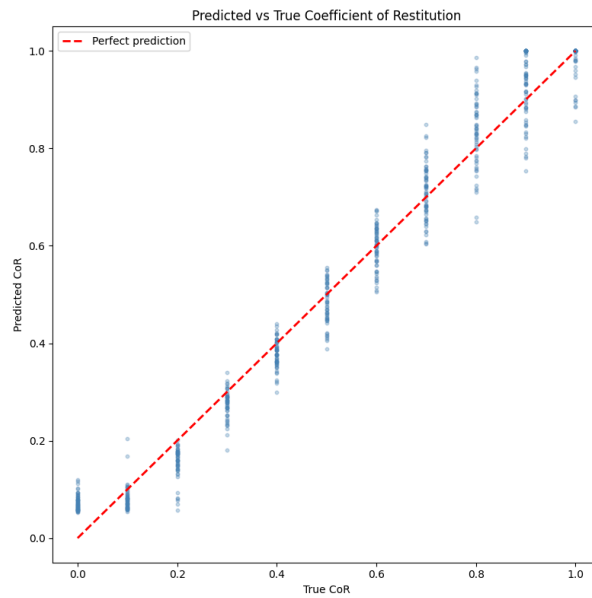
Although, ResNet18 + LSTM model had high accuracy for Unequal Mass data, it struggled in predicting Off Centre data's physics correctly.

3.2 Prediction Plots

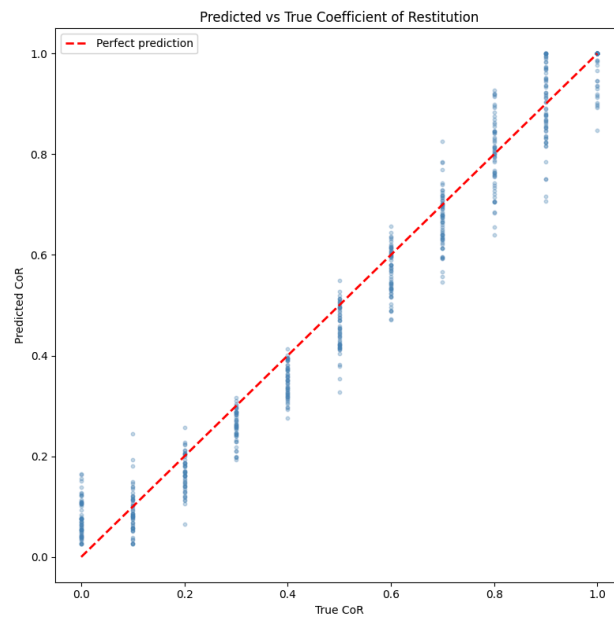
Architecture 1 - CNN + LSTM



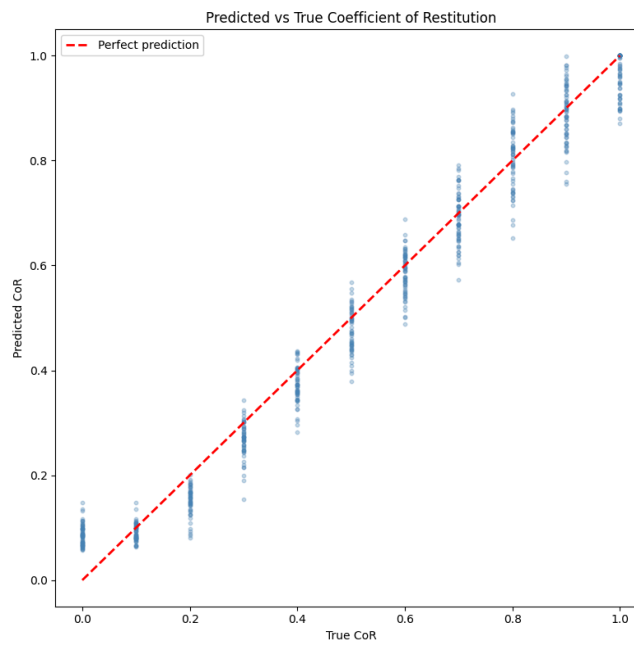
Architecture 2- CNN + Transformer



Architecture 3 - 3D CNN



Architecture 4 - ResNet18 + LSTM



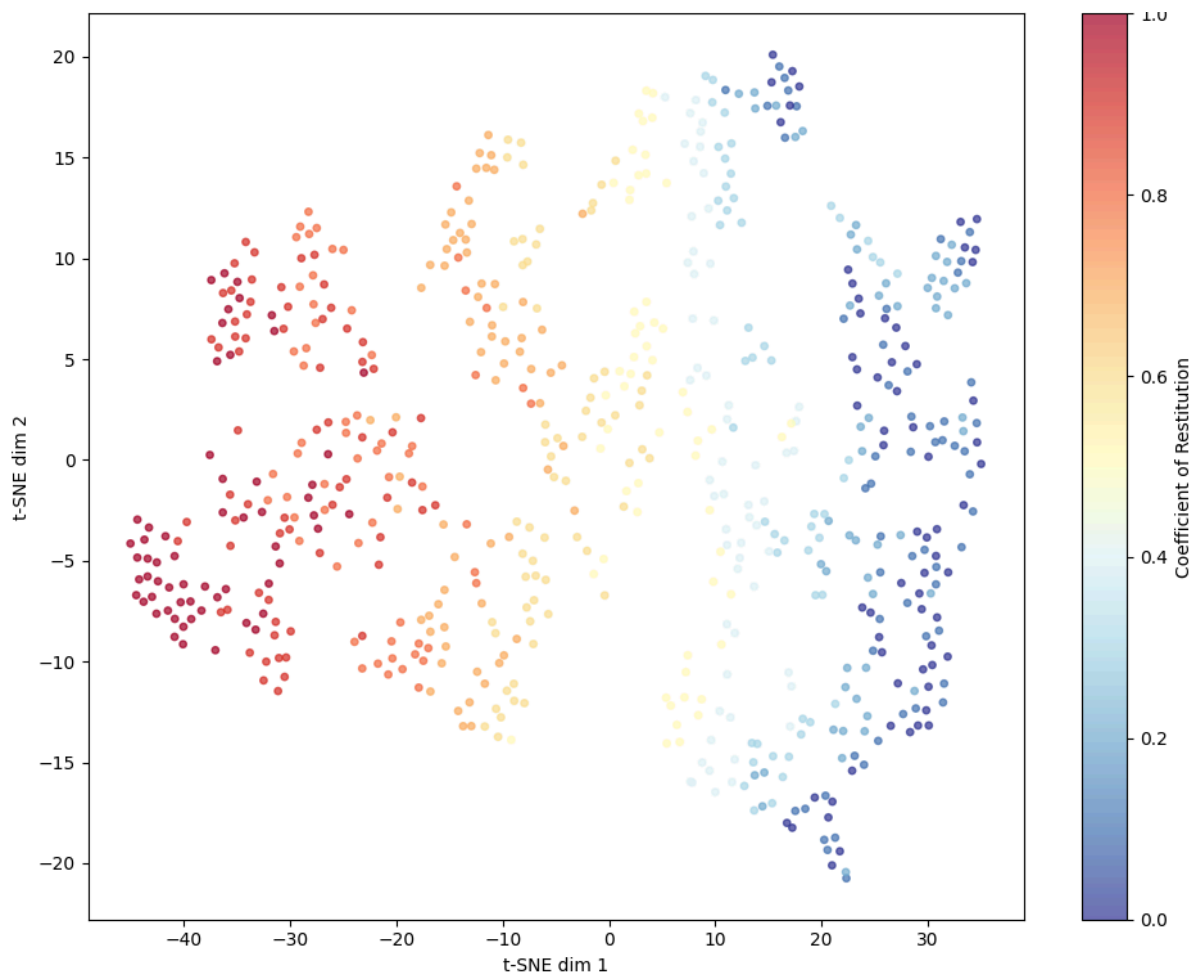
4. Physical Understanding Analysis

A series of tests were conducted to find if the models were able to understand the underlying physical dynamics of the system.

4.1 T-SNE Plots of the Latent Space

The latent space of models were extracted and then projected onto a 2 dimension plane. If the model had truly learnt the underlying physical dynamics, it should map the latent space such that the samples following the similar physics dynamics would be plotted closer and it would clearly transition from one dynamics to another.

For example, the t-SNE of Latent Space of Architecture 3 is shown below. We can clearly show that the samples which had similar CoR are mapped closer to each other and we can clearly see a transition from CoR = 0.0 to CoR = 1.0 (left to right) and hence we can say that the model has learnt to identify the physical dynamics of the system.



(The plots of t-SNE of latent space of other architectures can be found in their respective notebooks)

4.2 Momentum Conservation Probe

If the model has learnt the underlying physical dynamics of the system, then it should be able to follow the laws of conservation of momentum. Its prediction should follow the law of conservation of momentum which states that momentum of a system should be conserved.

Momentum of the system after the collision was calculated using the predicted CoR and final velocities and compared with initial momentum and these were the following results -

Models	Relative Momentum Error
CNN + LSTM	8.24%
CNN + Transformer	8.47%
3D CNN	6.87%
ResNet18 + LSTM	14.56%

It can be clearly seen that the models had very less relative error in predicting the momentum of the system. Architecture 3 (3D CNN) clearly stands out to be the best here, which is another metric which shows why Architecture is the best architecture of the four.

Other supporting metrics about the underlying physics dynamics can be seen in the models respective notebooks.

5. Conclusion

From the previous presented metrics and plots, it can be clearly seen that Architecture 3 is clearly the standout of the four architectures. Architecture 3 was not only able to perform well on in-distribution data but even excelled in out-of-distribution data, where all other architectures struggled. Architecture 3 was also able to follow the law of conservation of momentum, with least error among all architectures.

Hence, Architecture 3 - 3D CNN is the best architecture and I choose it as the final architecture for the task.